Programação Orientada a Objetos

Aula 05: Programação Orientada a Objetos em Java (continuação)

classes, objetos, métodos, construtores, sobrecarga, encapsulamento, modificadores de acesso, this, get e set

Prof. Fabricio Santana

fabricio.santana@idp.edu.br

www.linkedin.com/in/fabriciofsantana

O que estudamos na última aula?

- Plataforma java: JDK, JVM, jshell, java e javac
- Elementos da linguagem java:
- tipos primitivos, identificadores, variáveis, literais, operadores, palavras chaves, ...
- Entrada e saída
- Estruturas de controle:
  - seleção (if e switch) e iteração (for, while, do...while)
- Introdução a POO: classes, objetos, atributos e métodos

http://vevox.app

ID: 188-589-657

https://idp2.display.vevox.com#/present/721485/87WXDZ85NQFOYWI2DEH8

O que vamos estudar hoje?

- Continuação nos conceitos de Programação Orientada a objetos
- Classes, objetos e métodos
- Construtores (***new***)
- Sobrecarga de construtores
- Modificadores de acesso (***public***, ***private***, ***protected***)
- Métodos ***get*** e ***set***
- Encapsulamento
- Introdução ao tratamento de exceção
- Composição (has-a)
- Membros (atributos e métodos) de classe (***static***)

- **Atividade em sala (30 min)**:
  - em duplas, analisar as classes no repositório da disciplina em **poo/lectures/examples/08-classes**

Classes e Objetos

- **Classes**:
  - Tipo customizado (custom/reference type) que representa um conceito
  - Possui atributos e comportamento
    - atributos: variáveis de instância
    - comportamento: métodos
  - É um modelo/template

- **Objetos:**
  - Instância de uma classe
  - Um exemplo concreto

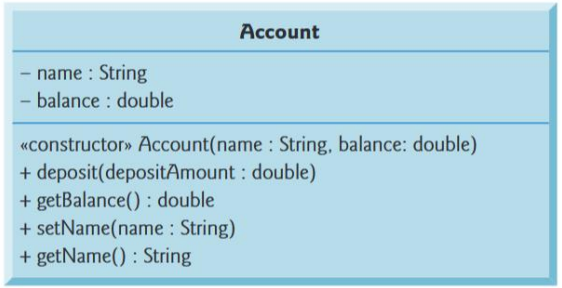

Fig. 3.10 | UML class diagram for Account class of Fig. 3.8.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Construtores

- Um objeto é criado por meio de um método especial, chamado ***construtor*** e uso da palavra reservada ***new***

- O construtor padrão tem o mesmo nome da classe e não tem parâmetros. Pode ser omitido no código, o compilador cria.

- Toda classe tem no mínimo um construtor

- É possível ter construtores customizados (com parâmetros)

```
public class Time1App {
    public static void main(String[] args) {
        Time1 time = new Time1();
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Construtores

- Acesso a membros de uma classe podem ser acessados dentro da própria classe com a palavra revervada ***this***

```
class SimpleTime {
    private int hour;
    private int minute;
    private int second;

    public SimpleTime(int hour, int minute, int second) {
        this.hour = hour;
        this.minute = minute;
        this.second = second;
    }
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Construtores customizados

- É possível desenvolver construtores customizados com a implementação de como um objeto será construído

- ***Overload constructors:*** simplesmente criar um declaração de construtor com o mesmo nome da classe que receba parâmetros

```java
public class Time2 {
    private int hour; // 0 - 23
    private int minute; // 0 - 59
    private int second; // 0 - 59

    public Time2(){
        this(0, 0, 0);
    }

    public Time2(int hour){
        this(hour, 0, 0);
    }

    public Time2(int hour, int minute){
        this(hour, minute, 0);
    }

    public Time2(int hour, int minute, int second){
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Acesso a membros de uma classe

- Modificadores de acesso:
  - Controla quem pode acessar atributos e métodos da classe
    - ***public:*** pode ser chamado por qualquer um
    - ***private:*** só podem ser chamados por métodos da mesma classe
    - ***proteced:*** só podem ser chamados por classes do mesmo pacote
- Métodos get e set são usados para controlar acesso a atributos privados da classe

```
public class MemberAccessTest {
    public static void main(String[] args) {
        Time1 time = new Time1();

        time.hour = 7; // error: hour has private access in Time1
        time.minute = 15; // error: minute has private access in Time1
        time.second = 30; // error: second has private access in Time1
    }
}
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Métodos *get* e *set*

- Métodos especiais que controlam o acesso aos atributos de uma classe/objeto
  - *set*: ***mutator methods***
  - get: ***acessor methods***

- ***Por que usar sets e gets ao invés de deixar o atributos públicos:***

```
public void setSecond(int second){
    if (second < 0 || second >= 60) {
        throw new IllegalArgumentException("second must be 0-59");
    }

    this.second = second;
}

public int getHour(){
    return hour;
}

public int getMinute(){
    return minute;
}

public int getSecond(){
    return second;
}
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Exceção

- ***try...catch***
  - Mecanismos de tolerância a falha para evitar comportamento indesejado
  - Uma exceção pode ser disparada com a palavra reservada ***throw***

```
try {
    time.setTime(99, 99, 99);
}
catch (IllegalArgumentException e) {
    System.out.printf("Exception: %s%n%n", e.getMessage());
}
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Composição

- Uma classe pode ter como atributo referências para outras classes

- Relacionamento ***has-a***

```java
public class Employee {
    private String firstName;
    private String lastName;
    private Date birthDate;
    private Date hireDate;

    public Employee(String firstName, String lastName, Date birthDate, Date hireDate) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.birthDate = birthDate;
        this.hireDate = hireDate;
    }

    public String toString() {
        return String.format("%s, %s  Hired: %s  Birthday: %s", lastName, firstName, hireDate, birthDate);
    }
}
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Membros de classe estáticos

- Cada objeto tem a cópia de seus próprios dados (variáveis de instância da classe)
- ***static field*** ou ***class variable*:** permite compartilhar dados entre todas os objetos de uma classe
- O escopo da variável de classe é a própria classe
- Um método estático não pode acessar uma variável de instância ou método de instância
- Em um método estático não é possível usar a palavra reservada ***this***

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Membros de classe estáticos

- Podem ser chamados diretamente da classe, ou seja, sem a necessidade de instanciar um objeto
  - Declarados com a palavra reservada ***static***
  - Forma de chamar: NomeDaClasse.metodoEstático()

```java
public class Time1 {
    private int hour; // 0 - 23
    private int minute; // 0 - 59
    private int second; // 0 - 59

    public void setTime(int hour, int minute, int second) {
        if (hour < 0 || hour >= 24 || minute < 0 || minute >= 60 || second < 0 || second >= 60) {
            throw new IllegalArgumentException("hour, minute and/or second was out of range");
        }

        this.hour = hour;
        this.minute = minute;
        this.second = second;
    }

    public String toUniversalString() {
        return String.format("%02d:%02d:%02d", hour, minute, second);
    }

    public String toString() {
        return String.format("%d:%02d:%02d %s", ((hour == 0 || hour == 12) ? 12 : hour % 12), minute, second, (hour < 12 ? "AM" : "PM"));
    }
}
```

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Membros de classe estáticos

- Podem ser chamados diretamente da classe, ou seja, sem a necessidade de instanciar um objeto
  - Declarados com a palavra reservada ***static***
  - Forma de chamar: NomeDaClasse.metodoEstático()

- Variáveis de instância ***final***

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

3ª Leitura

- BUDD, Timothy A. Thinking object-oriented**.** In: ____. *An introduction to object-oriented programming.* 3. ed. Boston: Addison-Wesley, 2002. Cap. 1. ISBN 0-201-76031-2.

- Prazo: 17/03/2025 às 8h15

- **Relembrando**: 2ª atividade para 19/03/2025 até 8h15

INSTITUTO BRASILEIRO DE ENSINO, DESENVOLVIMENTO E PESQUISA

fabricio.santana@idp.edu.br
www.linkedin.com/in/fabriciofsantana